Static HTML Website Hosting using Docker and Nginx on AWS EC2

This project demonstrates hosting a static HTML website inside a Docker container using the official Nginx image on an AWS EC2 Ubuntu instance. The website is accessed through the EC2 public IP using Docker port mapping. The project covers EC2 instance setup, Docker installation, Docker image creation, container execution, and web hosting using Nginx.

Aim

To create and host a static HTML website inside a Docker container using the official Nginx image on an AWS EC2 Ubuntu server and access it through the EC2 public IP using port mapping.

Tools / Requirements

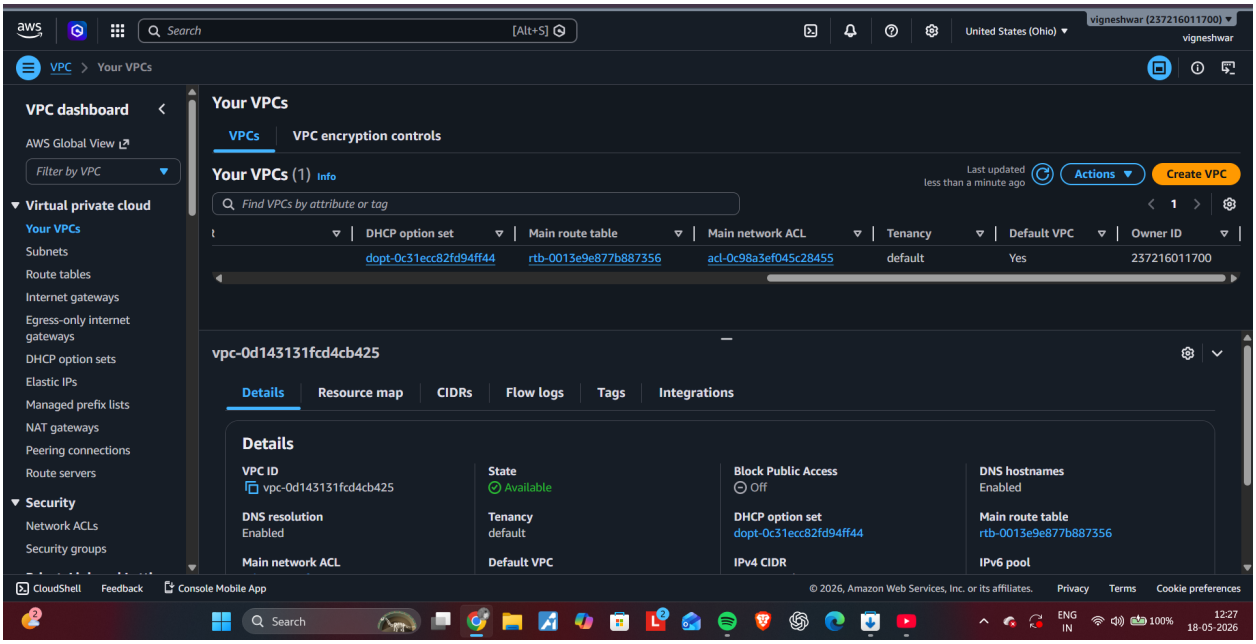
AWS EC2 Ubuntu Server
Docker
Nginx Docker Image
Linux Terminal
Internet Browser
Security Group Rules

Architecture

User Browser → EC2 Public IP → Docker Container → Nginx → Static HTML Website

Step 1: Create EC2 Instance

Created an AWS EC2 Ubuntu t2.micro instance with HTTP and SSH ports enabled.

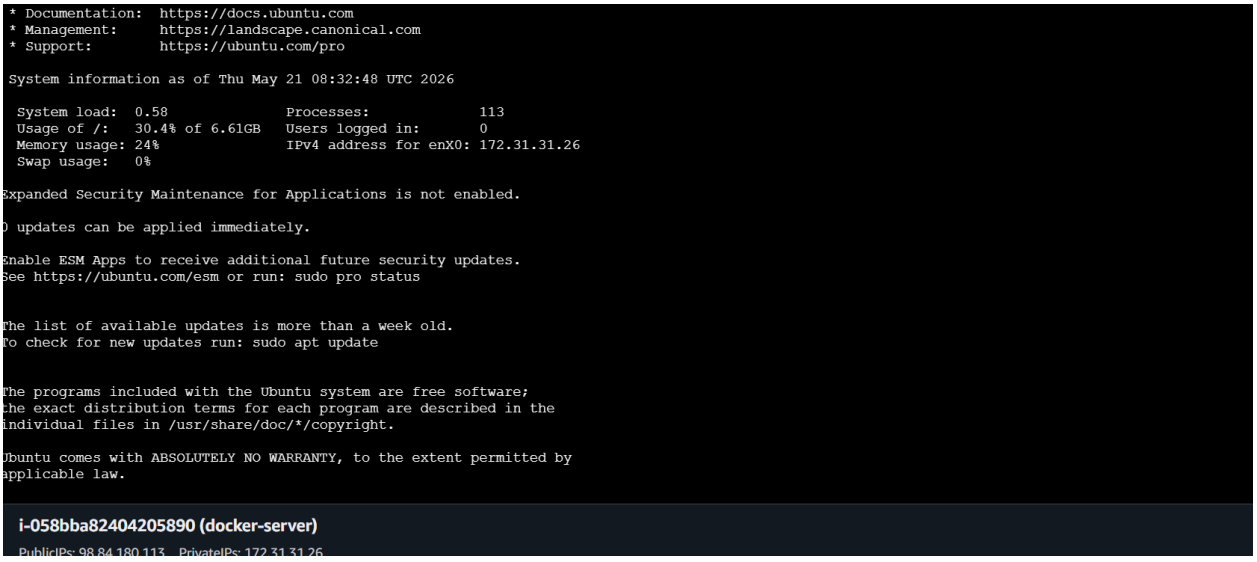


Step 2: Connect to EC2 Instance

Connected to the Ubuntu server using EC2 Instance Connect terminal.

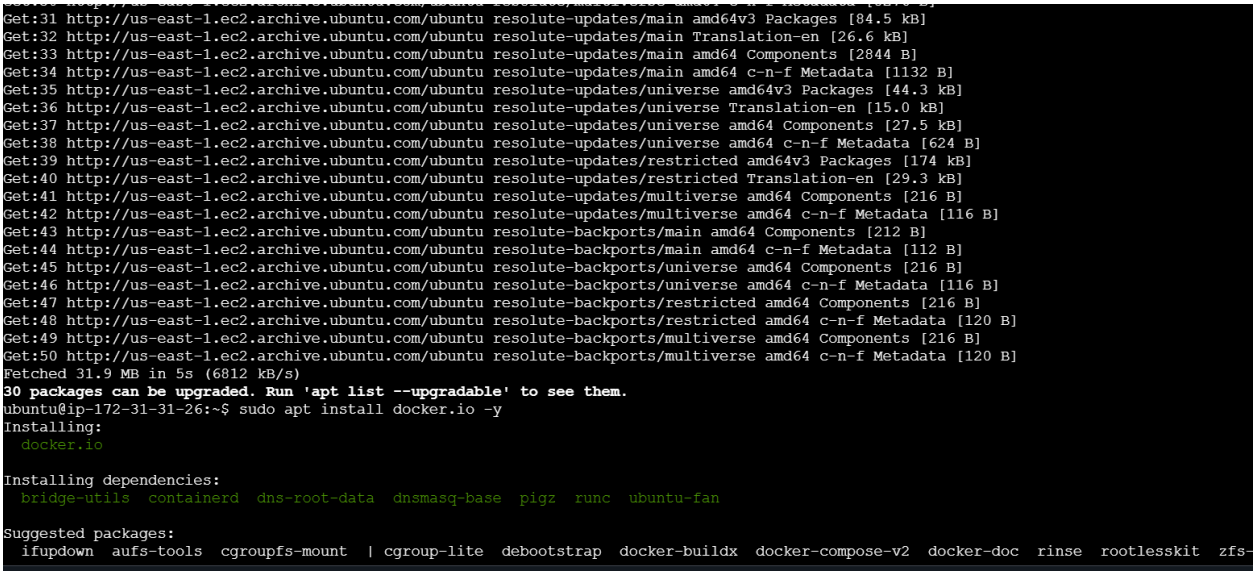
Step 3: Update Ubuntu Packages

Updated Ubuntu repositories using sudo apt update command.



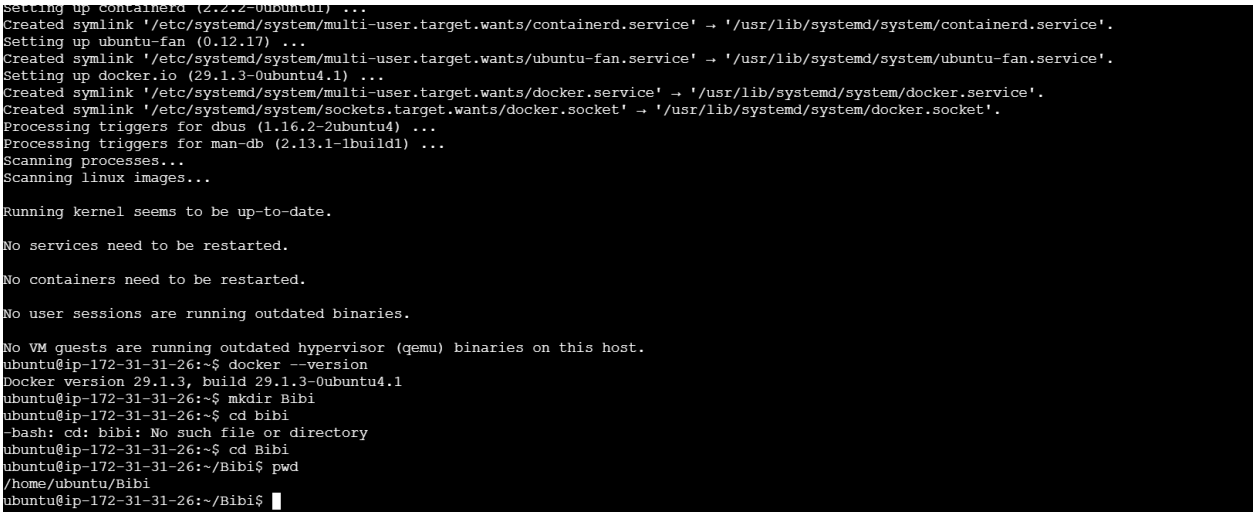
Step 4: Install Docker

Installed Docker on the Ubuntu server and verified the Docker version.



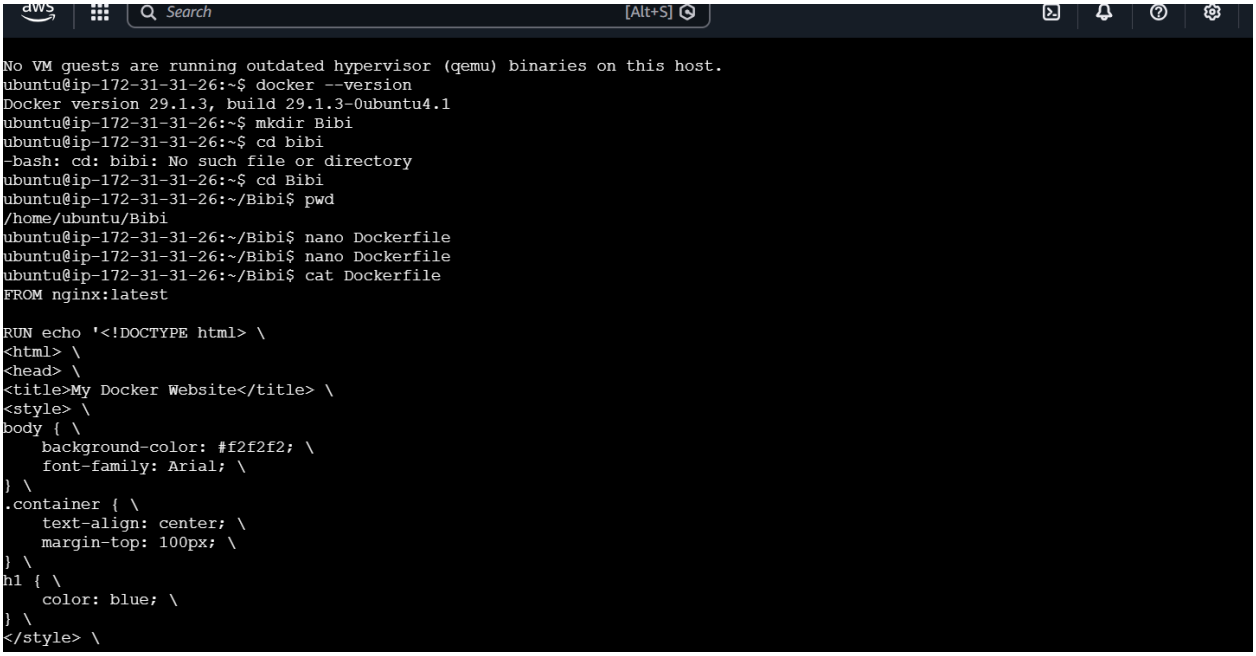
Step 5: Create Project Folder

Created the project directory named Bibi and navigated into it.



Step 6: Create Dockerfile

Created the Dockerfile containing the Nginx base image and static HTML content.



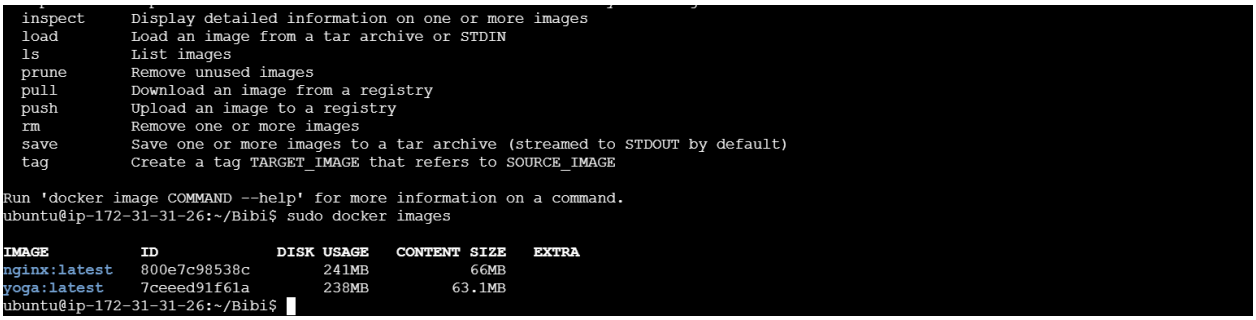
Step 7: Build Docker Image

Built the Docker image using the Dockerfile and tagged it as yoga:latest.

```
e2dc6cdcd988: Pull complete
6735e617770d: Pull complete
Digest: sha256:800e7c98538c6bf725f5177e841aa720ae0ed1c378bbea368b6330bfe18a36b3
Status: Downloaded newer image for nginx:latest
 ---> 800e7c98538c
Step 2/3 : RUN echo '<!DOCTYPE html> <html> <head> <title>My Docker Website</title> <styl
    text-align: center;     margin-top: 100px; } h1 {     color: blue; } </style> </head>
running inside EC2 container</p> </div> </body> </html>' > /usr/share/nginx/html/index.ht
 ---> Running in e989ca7cb07f
 ---> Removed intermediate container e989ca7cb07f
 ---> fae5b1dd094f
Step 3/3 : EXPOSE 80
 ---> Running in abfb39ed9f51
 ---> Removed intermediate container abfb39ed9f51
 ---> 7ceeed91f61a
Successfully built 7ceeed91f61a
Successfully tagged yoga:latest
ubuntu@ip-172-31-31-26:~/Bibi$
```

Step 8: Verify Docker Images

Verified the created Docker images using docker images command.



```
  inspect     Display detailed information on one or more images
  load        Load an image from a tar archive or STDIN
  ls          List images
  prune       Remove unused images
  pull        Download an image from a registry
  push        Upload an image to a registry
  rm          Remove one or more images
  save        Save one or more images to a tar archive (streamed to STDOUT by default)
  tag         Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE

Run 'docker image COMMAND --help' for more information on a command.
ubuntu@ip-172-31-31-26:~/Bibi$ sudo docker images

IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
nginx:latest   800e7c98538c       241MB          66MB
yoga:latest    7ceeed91f61a       238MB        63.1MB
ubuntu@ip-172-31-31-26:~/Bibi$
```

Step 9: Run Docker Container

Started the Docker container with port mapping using docker run command.

```
Usage:  docker image COMMAND

Manage images

Commands:
  build       Build an image from a Dockerfile
  history     Show the history of an image
  import      Import the contents from a tarball to create a filesystem image
  inspect     Display detailed information on one or more images
  load        Load an image from a tar archive or STDIN
  ls          List images
  prune       Remove unused images
  pull        Download an image from a registry
  push        Upload an image to a registry
  rm          Remove one or more images
  save        Save one or more images to a tar archive (streamed to STDOUT by default)
  tag         Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE

Run 'docker image COMMAND --help' for more information on a command.
ubuntu@ip-172-31-31-26:~/Bibi$ sudo docker images

IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
nginx:latest   800e7c98538c       241MB          66MB
yoga:latest    7ceeed91f61a       238MB        63.1MB
ubuntu@ip-172-31-31-26:~/Bibi$ sudo docker run -d -p 80:80 --name eni yoga
7d3e3e4dcca0efaff8852177c29f2a0fafdea45eb2021e3db7d7f08ea061ecc1
ubuntu@ip-172-31-31-26:~/Bibi$
```

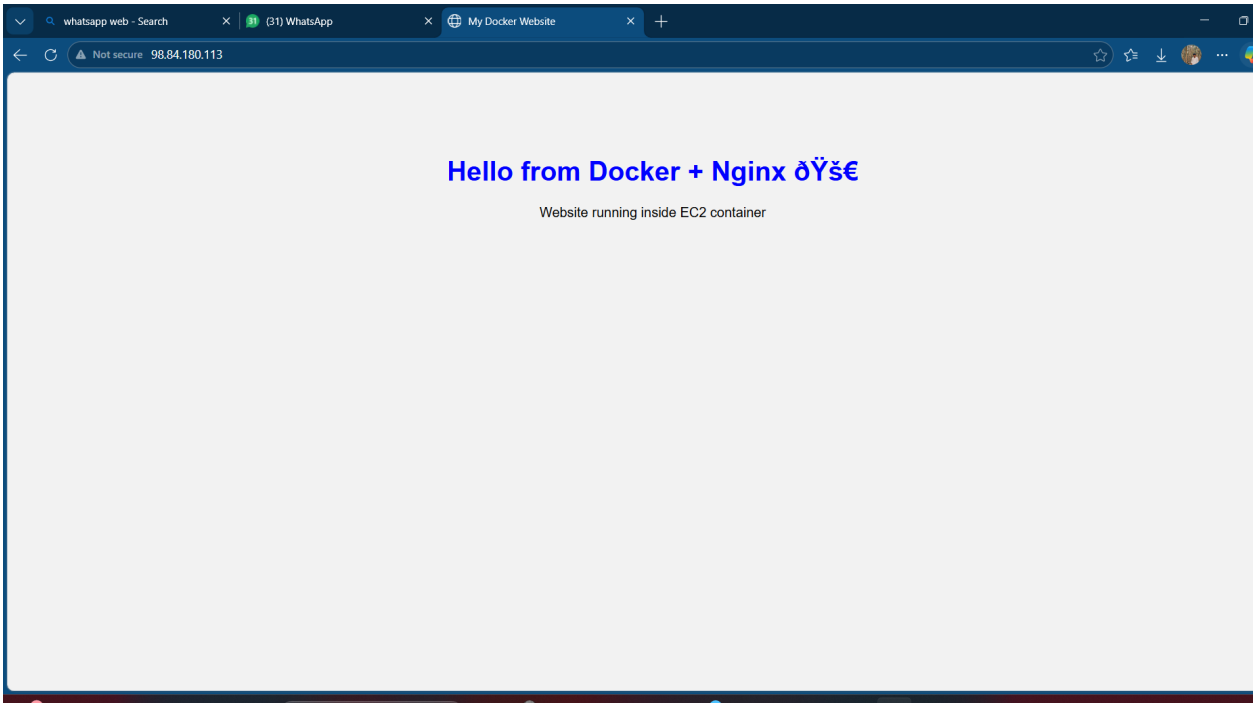
Step 10: Verify Running Container

Verified the running Docker container using docker ps command.

```
Manage images

Commands:
  build       Build an image from a Dockerfile
  history     Show the history of an image
  import      Import the contents from a tarball to create a filesystem image
  inspect     Display detailed information on one or more images
  load        Load an image from a tar archive or STDIN
  ls          List images
  prune       Remove unused images
  pull        Download an image from a registry
  push        Upload an image to a registry
  rm          Remove one or more images
  save        Save one or more images to a tar archive (streamed to STDOUT by default)
  tag         Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE

Run 'docker image COMMAND --help' for more information on a command.
ubuntu@ip-172-31-31-26:~/Bibi$ sudo docker images

IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
nginx:latest   800e7c98538c        241MB           66MB
yoga:latest    7ceeed91f61a        238MB         63.1MB
ubuntu@ip-172-31-31-26:~/Bibi$ sudo docker run -d -p 80:80 --name eni yoga
7d3e3e4dcca0efaff8852177c29f2a0fafdea45eb2021e3db7d7f08ea061ecc1
ubuntu@ip-172-31-31-26:~/Bibi$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                   NAMES
7d3e3e4dcca0   yoga      "/docker-entrypoint.…"   38 seconds ago   Up 38 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp   eni
ubuntu@ip-172-31-31-26:~/Bibi$
```

Step 11: Access Website

Accessed the hosted static website using the EC2 public IP address.



Port Mapping Explanation

Command	Description
-p 80:80	Maps EC2 port 80 to Docker container port 80
First 80	Host / EC2 Port
Second 80	Docker Container Port

Important Docker Commands

```
docker images
docker ps
docker stop eni
docker rm eni
docker rmi yoga:latest
```

Result

A static HTML website was hosted successfully using Docker and the Nginx image on AWS EC2. The website was accessed successfully through the EC2 public IP using HTTP port 80.

Conclusion

This project demonstrates Docker containerization and Nginx web hosting on AWS EC2. The implementation helped in understanding EC2 instance management, Docker installation, Docker image creation, container execution, port mapping, and static website hosting using Nginx.